

ValuNest

THE HOUSE PRICE PREDICTOR

B.Tech Project Report — CSE

GITAM School of Technology, Hyderabad

Guided by Dr. Arshad Ahmad Khan Mohammad

Akiranandan Reddy G | Harish Kumar G | Tejas Reddy I

ML · Flask · scikit-learn

Introduction & Problem Statement

THE PROBLEM

Property markets are unpredictable and complex for average consumers.

Traditional valuations rely on anecdotal knowledge or costly appraisals.

No accessible, data-driven tool for the Indian real estate market.



THE SOLUTION

ValuNest democratizes property price estimation via predictive modeling.

Uses Multiple Linear Regression on historical data from 6 major Indian cities.

Delivers real-time price estimates through a user-friendly Flask web app.

Abstract

ValuNest is an end-to-end machine learning web application for the Indian real estate market.

Provides reliable, data-driven house price estimates using historical data from major metro cities.

Built with Python, Flask, and scikit-learn — featuring a user-friendly HTML/CSS frontend.

Users can explore property prices by features (area, bedrooms, amenities, location).

Supports budget-based property browsing and a simulated booking & payment flow.

Project Objectives



Accurate Valuation

Develop a reliable ML model to estimate residential property prices based on structured attributes and geographical data.



User-Centric Discovery

Enable users to search for existing listings aligned with their specific budget constraints seamlessly.



Secure Access

Implement a secure authentication system using SQLite and password hashing to protect user sessions.



End-to-End Workflow

Provide a complete journey: from price prediction or budget filtering to simulated payment and booking confirmation.

System Architecture

LAYER 1

Frontend (Presentation)

HTML5 · CSS3 · Jinja2

Handles user input, displays predictions, renders property listings, manages auth & payment UI.

LAYER 2

Backend (Application)

Python 3 · Flask · Werkzeug

Manages HTTP routing, user sessions, form processing, interfaces ML model and database.

LAYER 3

Data & ML Layer

SQLite · pandas · NumPy · scikit-learn · Pickle

Stores credentials, ingests CSV data, performs feature engineering, serves predictions via .pkl model.

Dataset Description

Coverage: 6 Major Indian Metro Cities

Bangalore

Mumbai

Delhi

Hyderabad

Chennai

Kolkata

Key Features

Feature Type	Features
Target Variable	Price (market value in INR)
Continuous	Area (sq ft), Number of Bedrooms
Boolean / Amenities	CarParking, LiftAvailable, Gymnasium, SwimmingPool, 24X7Security, PowerBackup, ClubHouse
Categorical	City (one-hot encoded — 6 metro cities)
Preprocessing	One-hot encoding for City; dummy variable trap avoided by dropping first category

Machine Learning Model

Algorithm

Multiple Linear Regression

Selected for interpretability and efficiency in modeling the linear relationship between property features and market price.

Mathematical Formula

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Y = Predicted Price β_0 = Baseline price

$\beta_1.. \beta_n$ = Feature weights ϵ = Error term

Training Workflow

1

Data Ingestion
(pandas CSV load)



2

City Encoding
(One-Hot / Dummies)



3

Train / Test Split
(80% / 20%)



4

Model Fitting
(LinearRegression)

Results & Evaluation

R^2 Score

0.15

15% of price variance explained by the model's features (Area, Bedrooms, Amenities, City).

Mean Absolute Error

₹77,71,761

Average deviation between predicted and actual market prices — indicating the model's typical financial error margin.

Top Feature Weights (Model Coefficients)

Web Application

Authentication System

Users register and log in before accessing core features. Passwords hashed via Werkzeug's `generate_password_hash` — raw passwords are never stored in SQLite.

Prediction Module

Flask route captures property specs, builds a NumPy array matching the model input shape (with city dummy variables), and calls `model.predict()`.

Filter Module

Users browse `merged_files.csv` by setting a budget. App filters the pandas DataFrame and returns matching properties.

Payment & Booking

Simulated checkout using Flask sessions to pass pricing and user context securely between routes. Supports both prediction and filter workflows.

Advantages & Limitations

✓ ADVANTAGES



Accessible code-free interface for predictive analytics



Instantaneous price approximations — faster than manual research



Modular design: ML training is separate from the web app



Security: route protection + password hashing implemented

✗ LIMITATIONS



Location only at City level — misses sub-locality price premiums



Linear Regression may miss complex non-linear housing relationships



Static CSV dataset — no real-time data ingestion pipeline



Pickle serialization poses security/versioning risks in production

Web Page UI



Login Screen

The screenshot shows a dark-themed login form titled "Welcome back" with the subtitle "Sign in to your account". It features input fields for "Email address" (containing "john@demo.com") and "Password" (containing "New password"). Below the password field is a "Sign in" button and a link to "Don't have an account? Register here".



City Selection

The screenshot shows a dark-themed city selection screen titled "Which city?". It includes a subtitle "Select the city which you want to check property price". There are six buttons with icons and labels: "Bangalore", "Chennai", "Delhi", "Hyderabad", "Kolkata", and "Mumbai". At the bottom are "Back" and "Next" buttons.



Property Details

The screenshot shows a dark-themed property details form titled "Property details" with the subtitle "Fill the details to get your price estimate". It includes input fields for "Area(sqft)" (1450), "Bedrooms" (4+2), "Bathrooms" (3), "Property type" (New), "Car parking" (10), and "Furnishing" (Available). There are checkboxes for "Furnished", "Semi-furnished", "Not furnished", "Power backup", and "Close house". At the bottom are "Back" and "Get price estimate" buttons.



Estimated Price

The screenshot shows a dark-themed estimated price screen titled "Estimated Price" with the subtitle "Estimated price of your property". It displays the estimated price as "₹ 9,726,917". Below this are two boxes showing the price range: "₹ 8,546,754" and "₹ 11,062,075". At the bottom are "Try again" and "Proceed to payment" buttons.

Code Walkthrough

train_model.py — Core ML Logic

```
import pandas as pd; from sklearn.linear_model import LinearRegression
data = pd.read_csv('merged_files.csv')
data = pd.get_dummies(data, columns=['City'], drop_first=True)
X_train, X_test, y_train, y_test = train_test_split(features, target, test_size=0.2)
model = LinearRegression(); model.fit(X_train, y_train)
```

app.py — Flask Routing & Payment Logic

```
@app.route('/payment', methods=['GET', 'POST'])
@login_required
def payment():
    if session.get('payment_from') == 'filter':
        price = session.get('selected_price', 0)
        return render_template('payment.html', price=price, city=session.get('city', ''))
```

Conclusion

- ▶ ValuNest demonstrates practical ML integration within a full-stack Flask web application.
- ▶ Bridges the gap between complex real estate analytics and everyday users.
- ▶ Current R^2 of 0.15 highlights room for improvement via advanced ML models (Random Forest, Gradient Boosting).
- ▶ Future scope: Real-time data feeds, sub-locality encoding, cloud deployment.
- ▶ A robust, functional foundation for a commercial property valuation platform.

ValuNest

*ML · Flask
scikit-learn
HTML/CSS*